ECDLP methods priced in one unit: bit operations

research/bitop_unit_20261016 · 2026-10-16 · aburan28/crypto branch `claude/bitop-work-unit`

What this is

Correction to the unit's interpretation

The protocol initially described `bitop/v1` as a count of two-input Boolean gates. That wording overstated what its formulas establish. The formulas, recorded counts, and numerical results are unchanged; `bitop/v1` is a reproducible accounting convention whose primitive weights are expressed as gate-equivalent model units. They are not physical gate counts, circuit lower bounds, or costs of a synthesized implementation. The instruction audit diagnoses possible missing counters; it does not validate formula weights as circuit costs or prove complete instrumentation. The correction is also recorded in `PROTOCOL.md`.

Until now, the repository and its siblings (`aburan28/cryptanalysis`, cairn) charged ECDLP work in at least six units:

- group-addition equivalents, with non-group work converted by one host's nanosecond ratios;
- solver work priced by wall time, or not at all;
- `full_prices.json` ratios;
- reference picoseconds;
- Callgrind instructions;
- wall-clock online speedups.

No two of these can be divided, so an exponent fitted in one is not an exponent in another. Worse, work without a conversion ratio was charged **zero**: Buchberger, SAT and PDP solver work, and linear algebra on any curve without a calibration row. Every index-calculus figure was therefore a lower bound.

`bitop/v1` (`docs/units/BITOPS.md`, `src/cryptanalysis/bitops.rs`) is a reproducible accounting convention with frozen gate-equivalent formula weights that depend only on operand width. These weights are not physical gate counts, circuit lower bounds, or costs of a synthesized implementation:

- field product, squaring and inversion in $\mathbb{F}_p$, $\mathbb{F}_{2^n}$ and $\mathbb{F}_{p^4}$;
- affine addition and doubling;
- multiply, add and inversion modulo r;
- a 64-bit word operation;
- a hash probe.

A run's price is an exact integer under the convention and does not depend on the host. `ecbench::work` prices any record from its own counts. A record whose counters include one with no rule, or whose code is known to do work no counter sees, is **incomplete**, and its bound in this unit is inadmissible.

`ecbench audit-ir` replays a run under Callgrind and compares instructions per model unit. A ratio far above rho's on the same curve diagnoses possible work its counters miss. This audit does not validate the formula weights as circuit costs and does not prove that all work is counted.

The protocol (`PROTOCOL.md`) was frozen before any session; its deviations and this post-study correction to the unit's interpretation are listed there. The recorded counts, formulas, and numerical results are unchanged by the correction.

Counter coverage and instruction audit

Instrumentation was added to these solver and pipeline paths:

- F4, F5, inherited-F4, F6, Buchberger, crossbred, XL, FES, exhaustive and SAT now count their whole run;
- so do the descent oracles' system builds;
- so do the pipeline's Gauss scans and pivot inversions, relation rows, walk bookkeeping, table inserts, base indexes and canonicalisation tables.

New counters live in fields a replay of a committed record ignores, so every committed session still replays exactly. This was checked on nine sessions covering every engine family. A record is called complete only when its priced counters and the explicit known-unaccounted list are both empty; the audit is a diagnostic comparison and does not prove that every operation is counted.

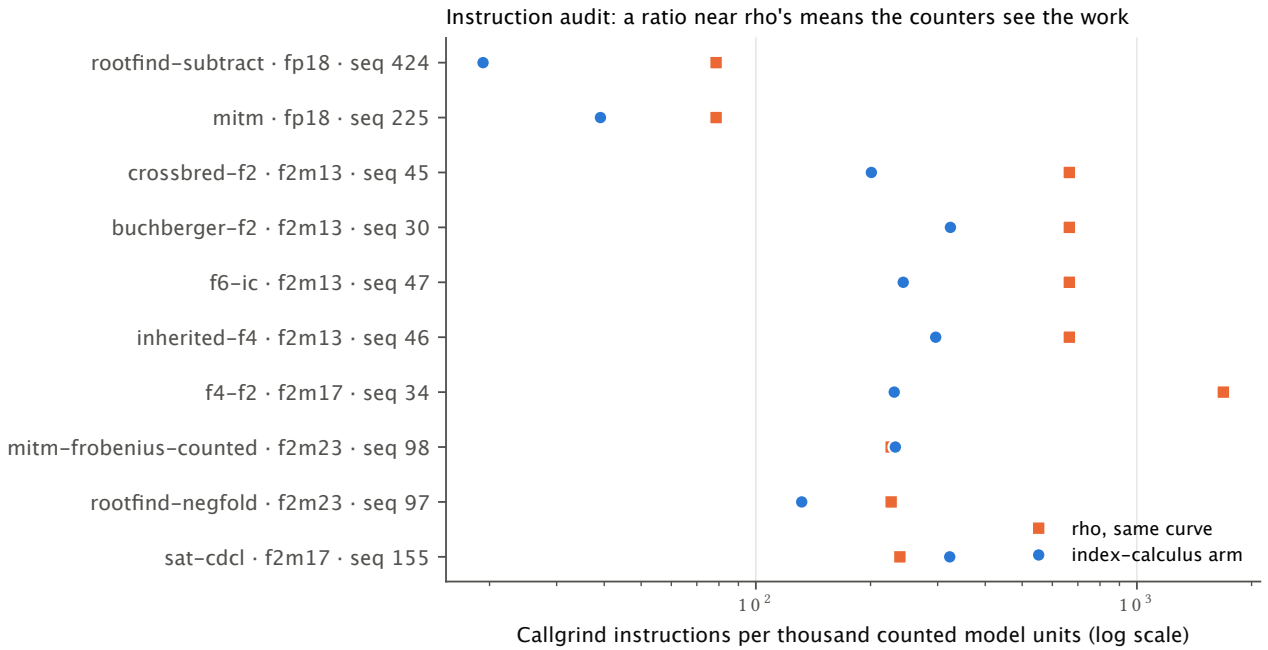


Figure 1: Instructions per thousand counted model units, each index-calculus arm against rho on the same curve, with the final binary (`audit/final-panel.json`). Every arm sits within 2 × of its rho, and most sit below it. This is diagnostic evidence, not proof of complete instrumentation.

seq	method · oracle / solver	curve	instr./kbitop	× floor	complete
155	ic.pipeline_counted · sat-cdcl	icv1-f2m17	322.71	$1.66{\cdot}10^{4}$	yes
159	rho.signed_frobenius_strong ·	icv1-f2m17	238.8	26.61	yes
97	ic.pipeline_full · pdp3-koblitz	icv1-f2m23	131.93	116.16	yes
98	ic.pipeline_full · mitm-frobenius-counted	icv1-f2m23	232.34	10.41	yes
100	rho.signed_frobenius_strong_full ·	icv1-f2m23	226.5	5.27	yes
101	rho.signed_frobenius_full ·	icv1-f2m23	267.6	5.38	yes
34	ic.pipeline · f4-f2	icv1-f2m17	230.75	$7.15{\cdot}10^{3}$	yes
46	ic.pipeline · pdp3-binary	icv1-f2m13	296.49	$1.44{\cdot}10^{5}$	yes
47	ic.pipeline · pdp3-binary	icv1-f2m13	243.65	$1.09{\cdot}10^{5}$	yes
30	ic.pipeline · buchberger-f2	icv1-f2m13	324.09	$6.78{\cdot}10^{4}$	yes
45	ic.pipeline · crossbred-f2	icv1-f2m13	200.97	$5.59{\cdot}10^{4}$	yes
26	rho.signed_frobenius_strong ·	icv1-f2m13	664.88	60.82	yes
225	ic.pipeline · mitm	icv1-fp18	39.1	10.27	yes
227	rho.negation ·	icv1-fp18	78.64	2.11	yes
424	ic.pipeline · rootfind-subtract	icv1-fp18	19.23	24.71	yes
420	rho.negation ·	icv1-fp18	92.49	1.63	yes
35	rho.negation ·	icv1-f2m17	1686.11	1.74	yes

Rho on the 13-bit $\mathbb{F}_{2^{17}}$ curve (seq 35) reads 1,686 instructions per kilo-bitop because its walk is about 100 steps, and the per-solve fixed overhead dominates. The F4 arm on the same curve is at 231.

What the old unit left out

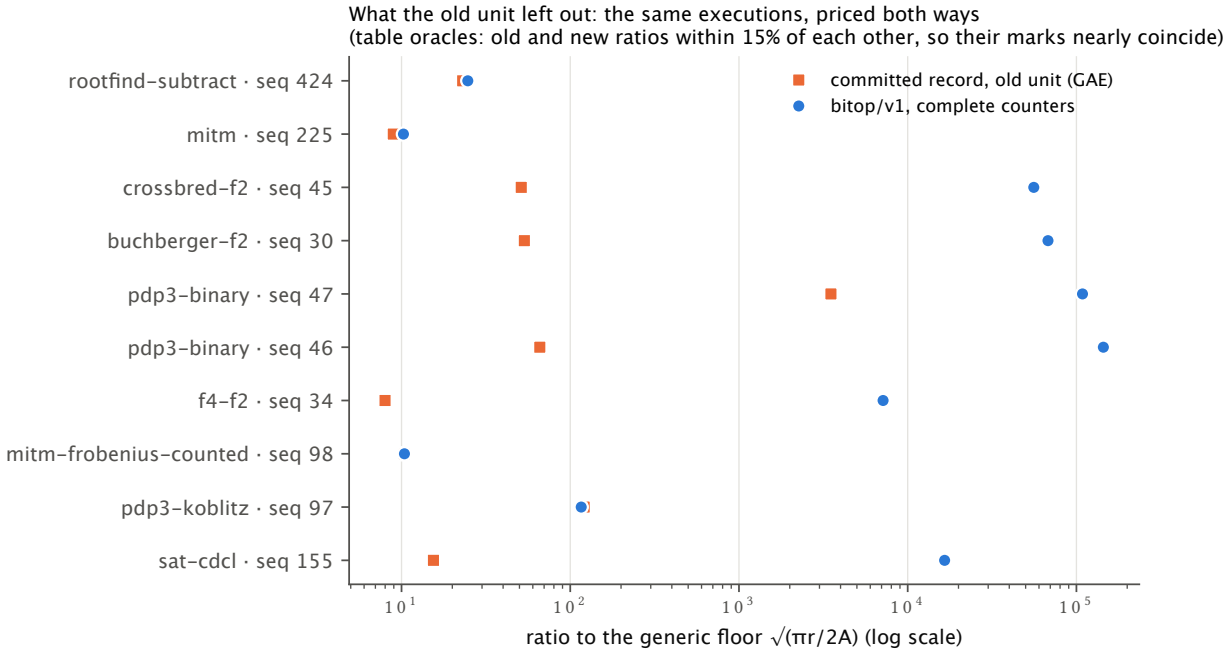


Figure 2: The same executions priced two ways: the committed record's ratio to the floor in group-addition equivalents, and the audit replay's in `bitop/v1` with complete counters.

- **Table oracles:** their old and new figures agree to within 15% (Frobenius-orbit MITM 0.2%, pdp3 root-finding 4%, prime root-finding 7%, prime MITM 15%).
- **Algebraic oracles:** they were understated by three orders of magnitude or more. Their solvers had been charged little or nothing:
  - F4 descent 8.0 → 7147 times the floor;
  - Buchberger 53 → 67837;
  - crossbred 51 → 55887;
  - SAT 15.5 → 16552;
  - inherited-F4 66 → 144448.
- **Signed-Frobenius rho:** its ratio roughly doubles (2.40 → 5.38), because its canonicalisations, a Frobenius normal form and two products modulo r per step, had been free.

Exponents with intervals

The prime ladder (`specs/prime-ladder.json`, session `ECBS1h3674c4a0b139`) has 11 prime-order curves, one at each bit size from 16 to 26, with 8 public targets each and 528 verified runs. Every arm's pricing is complete and every bound is admissible.

Each exponent is fitted twice. The **raw** exponent fits model units $= C \cdot r^{\alpha}$. The exponent **in additions** fits model units divided by one affine addition's model weight on each curve, which takes out the field arithmetic's $(\log r)^2$; there, generic theory says $\frac{1}{2}$. Intervals are two-stage cluster bootstrap, 95%, 2000 resamples.

arm	α raw	95%	α in additions	95%	× floor (26 bits)
rho-neg	0.566	[0.466, 0.66]	0.433	[0.32, 0.538]	1.42
bsgs-neg	0.607	[0.563, 0.646]	0.474	[0.436, 0.508]	1.26
kangaroo	0.605	[0.504, 0.704]	0.472	[0.37, 0.567]	2.84
ic-mitm-b32	0.898	[0.748, 1.029]	0.766	[0.6, 0.909]	29.97
ic-mitm-b128	0.388	[0.259, 0.495]	0.255	[0.112, 0.373]	36.62
ic-rootfind-m3-b32	1.058	[1.016, 1.097]	0.925	[0.873, 0.973]	111.97

Positive control: passed. Rho, BSGS and kangaroo all have $\frac{1}{2}$ inside their interval in additions, consistent with the expected exponent for these generic methods on this ladder.

Index calculus with a fixed base scales worse than rho at these sizes. The relation phase's exponent is about 1.0 for the 32-point base and about 0.83 for the 128-point base; the latter's low total exponent is its fixed table set-up dominating small sizes. These are exponents of these **configurations**. An index calculus that grows its base with r is a different method and needs its own ladder.

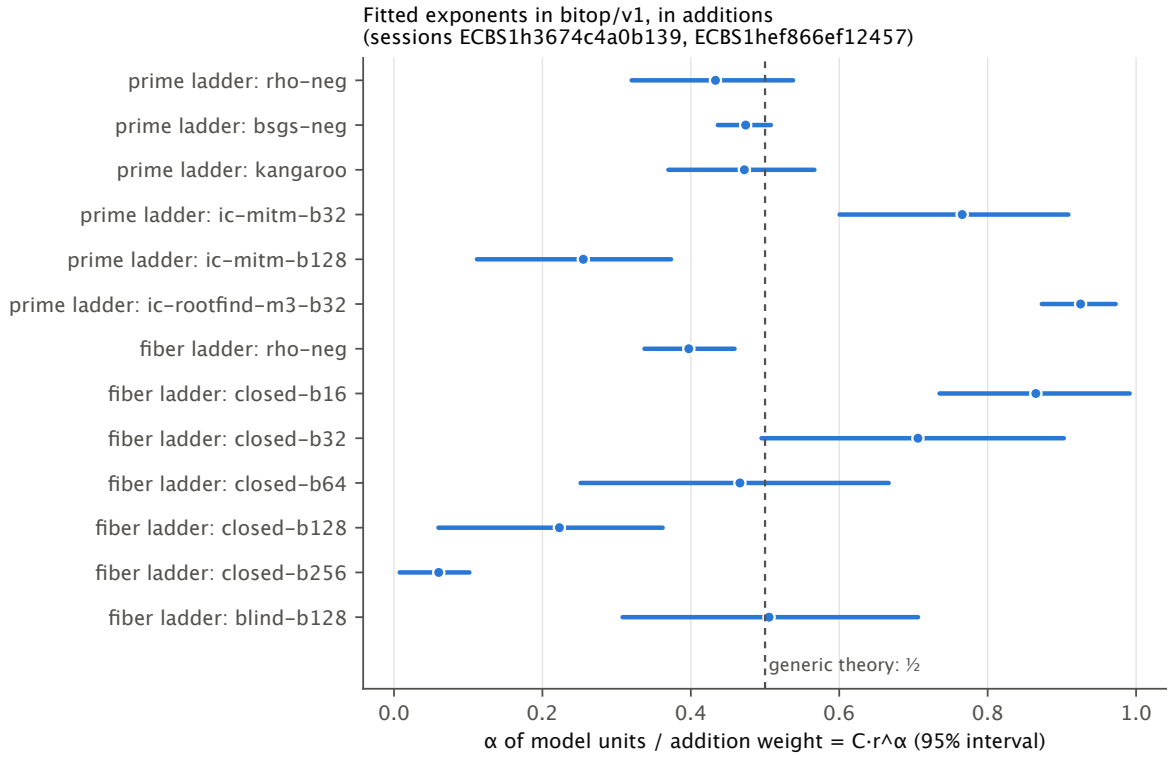


Figure 3: Exponents in additions with 95% intervals: prime ladder and fiber ladder.

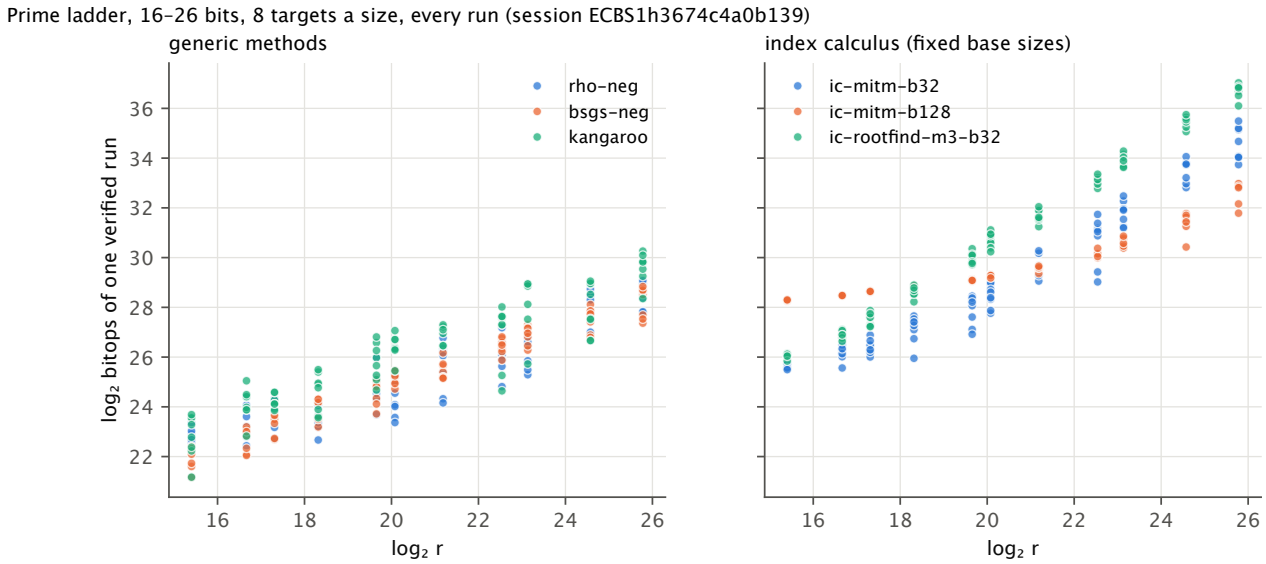


Figure 4: Every verified run of the prime ladder: $\log_2$ model units against $\log_2 r$.

Fiber ladder (`specs/prime-fiber-scaling.json`, session `ECBS1hef866ef12457`, 8 sizes). All seven arms are admissible. Rho's interval in additions, [0.338, 0.459], excludes $\frac{1}{2}$ by 0.041, inside the protocol's 0.05 tolerance. Its fixed jump-table set-up is a larger share at the small sizes, which pulls the fitted exponent down.

Koblitz curves ($a = 0$, $n = 13, 19, 23, 31$). Signed-Frobenius rho is at 7× the floor, and strong rho at 28.84×. Both are admissible. Their exponents in additions fall below $\frac{1}{2}$, as they should: the automorphism group they fold by has order $2n$, which grows with $\log r$. With 4 sizes, the intervals are wide.

How many digits an exponent has

An exponent is reported only to the width of its interval. Rho's interval in additions on the prime ladder is about ± 0.11 from 88 runs. The width scales as one over the square root of the runs, so a third decimal place (± 0.001) would take about 10^4 times as many runs, and widening the size range helps more than repeating sizes. The figures above carry exactly the digits their data support.

Algebraic oracles, constant only

These single-curve sessions are re-runs of committed specs on the instrumented binary. All are complete and admissible.

session · arm	× floor	95%
sat-count-qualified-ic-sat-xor	$4.59{\cdot}10^{4}$	[40139.1, 51569.02]
sat-count-qualified-ic-mitm	$1.84{\cdot}10^{3}$	[1843.1, 1843.49]
sat-count-qualified-ic-mitm-frob	121	[120.82, 121.17]
sat-count-qualified-rho-strong	28.12	[26.79, 29.55]
f4-k13-matrix-f4	$4.66{\cdot}10^{7}$	[40884480.34, 52416802.89]
f4-k13-matrix-f5	$3.52{\cdot}10^{7}$	[30906375.54, 39542900.91]
f4-k13-inherited-f4	$1.88{\cdot}10^{5}$	[168441.9, 209622.58]
f4-k13-f6-ic	$1.41{\cdot}10^{5}$	[126208.8, 156167.5]
f4-k13-rho-negation	10.27	[8.57, 12.13]
f4-k13-rho-strong	69.46	[66.98, 70.74]

In the yield sweep (`specs/yield-koblitz.json`, 4 sizes) every arm is admissible except the two SAT-descent arms. Those fit over 3 sizes and pass through a base builder and an elimination variant that are not yet instrumented; their bounds carry that and are not admitted.

Amortised: is there a number of targets at which index calculus wins?

`ecbench amortize` splits each run's model units into **reusable** and **per-target** work. The reusable part is the factor base and the oracle's tables. Each arm is then compared with batch rho, the Kuhn–Struik walk that keeps its table, whose cost grows like $\sqrt{T}$. Where a method wins at all it wins in a window of target counts, because batch rho's cost grows like $\sqrt{T}$ and the method's like T.

On the prime ladder, none of the three index-calculus arms has a window at any size. Each further target costs them 3.5–311 times what it costs rho (`amortize/`). The reason is that the single-target pipeline re-collects relations for every target. Only a target-blind variant such as `ic.shared_rank` could change that.

The archive

`ecbench work --summary` over every committed session (`archive/summary.json`) prices 20794 of 36793 verified runs completely: every generic-method run.

The rest are index-calculus records written before the new counters existed. They name exactly what they miss, and only a re-run can complete them, as this study's sessions were.

Limits

- **What the accounting convention leaves out:** memory, branches and allocation. Memory is reported beside the unit; a memory-cost model would be `bitop/v2`.
- **Conventions are frozen:**
  - schoolbook products, so Karatsuba would be `bitop/v2`;
  - a pentanomial reduction;
  - a 4κ hash probe.

  Every record keeps its primitive counts, so any later convention reprices the whole history.
- **Root-finding:** the arm batches its inversions but is charged an affine addition, inversion included, per candidate, the unit's convention. Its price is a convention-level model figure for that implementation; the instruction ratio is a separate diagnostic.
- **Isolation:** sessions ran at isolation L0 (`--cpus none`). Bit-operation counts do not depend on it; no wall time is used here.
- **No security claim:** nothing here concerns any deployed curve.

Provenance

- **Specs:** `specs/`.
- **Sessions:** `sessions/`, each verified with `ecbench verify --replay 2`.
- **Bounds:** `bounds/`, from `ecbench bound fit --unit bitop/v1`.
- **Audits:** `audit/`.
- **Amortisation:** `amortize/`.
- **Archive summary:** `archive/`.
- **Figures:** `figures/plots.py`, which reads only recorded values.